Fair Tree Fairshare Algorithm

Contents §

- Introduction
- Overview for End Users
- Algorithm
- Level Fairshare Calculation
- Ties
- sshare
- Configuration
- Important notes

Introduction §

Starting with Slurm 19.05, the Fair-share factor in the priority/multifactor plugin will default to the **Fair Tree** algorithm described on this page. The previously used Classic algorithm can be manually configured.

Fair Tree prioritizes users such that if accounts A and B are siblings and A has a higher fairshare factor than B, all children of A will have higher fairshare factors than all children of B.

Some of the benefits include:

- All users from a higher priority account receive a higher fair share factor than all users from a lower priority account.
- Users are sorted and ranked to prevent errors due to precision loss. Ties are allowed.

- Account coordinators cannot accidentally harm the priority of their users relative to users in other accounts.
- Users are extremely unlikely to have exactly the same fairshare factor as another user due to loss of precision in calculations.
- New jobs are immediately assigned a priority.

Overview for End Users §

This section is intended for non-admin users who just want to know how their fairshare factor is determined. Run `sshare -l` (lowercase "L") to view the following columns: `FairShare, Level FS`. Note that Level FS values are infinity if the association has no usage.

If an account has a higher Level FS value than any other sibling user or sibling account, all children of that account will have a higher FairShare value than the children of the other account. This is true at every level of the association tree.

The FairShare value is obtained by using the Fair Tree algorithm to rank all users in the order that they should be prioritized (descending). The FairShare value is the user's rank divided by the total number of user associations. The highest ranked user receives a 1.0 fairshare value.

If you (UserA) have a lower FairShare value than another user (UserB) and want to know why, find the first common ancestor account. At the level below the common ancestor, compare the Level FS value of your ancestor to the Level FS value of UserB's ancestor. Your ancestor has a lower Level FS value than UserB's ancestor. For information on how Level FS value is calculated, read the section about the Level FS equation.

For example, assume the association tree contains UserA and UserB as follows:

```
root => Acct1 => Acct12 => UserA
root => Acct1 => Acct16 => UserB
```

Acct1 is the first common ancestor of UserA and UserB. Check the Level FS values of Acct12 and Acct16. If UserB has a higher FairShare value than UserA, Acct16 has a higher Level FS value than Acct12.

The sections below contain more information about the algorithm, including how the final fairshare factor and the Level FS values are calculated.

Algorithm §

An equation is used to calculate a Level Fairshare value for each association, only considering the shares and usage of itself and its siblings. A rooted plane tree (PDF download), also known as a rooted ordered tree, is logically created then sorted by Level Fairshare with the highest values on the left. The tree is then visited in a depth-first traversal. Users are ranked in pre-order as they are found. The ranking is used to create the final fairshare factor for the user.

The algorithm performs a single traversal of the tree since all the steps can be combined. The basic idea is to set *rank* equal to the count of user associations then start at root:

- Calculate Level Fairshare for the subtree's children
- Sort children of the subtree
- Visit the children in descending order
  - If user, assign a final fairshare factor similar to (rank-- / user_assoc_count)
  - If account, descend to account

Level Fairshare Calculation §

The Level Fairshare equation is described below. Under-served associations will have a value greater than 1.0. Over-served associations will have a value between 0.0 and 1.0.

```
LF = S / U
```

LF
: is the association's Level Fairshare

S
: also known as Shares Norm, S is the association's assigned shares normalized to the shares assigned to itself and its siblings:
$S = Sraw_{self} / Sraw_{self+siblings}$

U
: also known as Effective Usage, U is the association's usage normalized to the account's usage: `U = Uraw_self / Uraw_self+siblings`

U and S are in the range `0.0 .. 1.0`. LF is in the range `0.0 .. infinity`.

Ties §

Ties are handled as follows:

- Sibling users with the same Level Fairshare receive the same rank
- A user with the same Level Fairshare as a sibling account will receive the same rank as its highest ranked user
- Sibling accounts with the same Level Fairshare have their children lists merged before descending

sshare §

sshare was modified to show the Level Fairshare value as `Level FS` when the `-l` (long) parameter is specified. The field shows the value for each association, thus allowing users to see the results of the fairshare calculation at each level.

Note: Norm Usage is not used by Fair Tree but is still displayed.

Configuration §

The following slurm.conf parameters are used to configure the Fair Tree algorithm. See slurm.conf(5) man page for more details.

PriorityType
: Set this value to "priority/multifactor".

PriorityCalcPeriod
: PriorityCalcPeriod is the frequency in minutes that job half-life decay and Fair Tree calculations are performed.

Important Notes §

- As the Fair Tree algorithm ranks all users, active or not, the administrator must carefully consider how to apply other priority weights in the priority/multifactor plugin. The *PriorityWeightFairshare* can be usefully set to a much smaller value than usual, possibly as low as 1 or 2 times the number of user associations.
- Fair Tree requires the Slurm Accounting Database to provide usage information and the assigned shares values.
- *scontrol reconfigure* does not cause the Fair Tree algorithm to run immediately, even if switching from a different algorithm. You may have to wait until the next iteration as defined by *PriorityCalcPeriod*.